From Large Language Models to Efficient System One Models via Adaptive Computation

Minh-Anh Nguyen[1]
[1]Nanyang Technological University, Singapore

arXiv:2611.16321v1 [cs.CL] 12 Nov 2026

Abstract

Transferring LLMs into System One decision models typically replaces the language modeling head with a decision head while retaining the full backbone, making every decision pay for all layers. Yet it remains unclear how much depth such decisions require. We find that decision information is concentrated in intermediate layers, with early layers building essential features and the final six layers removable without loss, while independently trained smaller models largely mimic the full model rather than developing specialized decision capabilities. These findings suggest that efficiency requires not simply smaller backbones, but shared computation across different depths. Building on this, we propose NESTONE, which jointly trains a full model with prefix submodels that share its final layers and decision head through self-distillation. A lightweight gate reads the hidden state at each exit to determine whether additional computation is needed, allowing one model to operate at multiple layers. Across seven decision benchmarks, our approach matches or outperforms independently trained models at every depth, while achieving better accuracy-efficiency trade-offs through adaptive computation.

1 Introduction

Dual-process theories distinguish fast, intuitive *System One* thinking from slow, deliberate *System Two* thinking (Kahneman, 2011). LLM-based agents increasingly require both. Alongside long chain-of-thought reasoning, they make frequent, lightweight decisions, such as routing requests, selecting tools, flagging unsafe prompts, and recognizing intents, often under tight latency constraints (TypeSafe AI, 2026). System One decision models address this need by fine-tuning an LLM to predict a distribution over candidate options at a single position, without generating tokens (InternLM Team, 2026; Li and Wang, 2026). For example, Intern-Decision-4B, built on Qwen3.5-4B (Qwen Team, 2026), achieves 90.0% macro accuracy across seven decision benchmarks. However, replacing the language modeling head with a decision head leaves the full backbone intact: every decision still traverses all 32 layers, incurring a latency of approximately 1.45 ms per layer at batch size one in our setting.

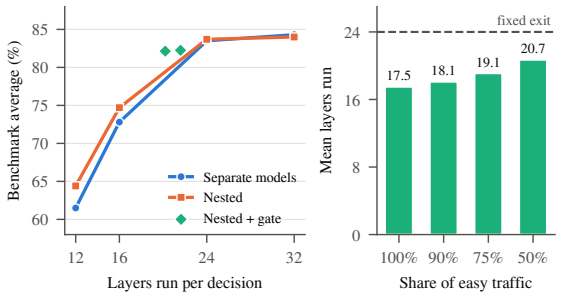

Figure 1: Left: Accuracy versus computation depth for independently trained models, and NESTONE w/o the stop gate. Right: Average depth of the gate versus the share of easy decisions; on the four largest benchmarks, accuracy stays within 0.1 points of the fixed 24-layer exit. All models use Qwen3.5-4B and the same training data; models are averaged over 3 seeds.

This raises a fundamental question: *how much computational depth does a System One decision actually require?* Fine-tuning is necessary to equip an LLM with decision capabilities that its base model lacks (Section 3), but whether all inherited layers are necessary remains unclear. A natural way to reduce inference cost is to truncate the backbone, trading computational depth for potentially lower accuracy. As shown in the left panel of Figure 1, this trade-off is highly non-uniform: reducing the model from 32 to 24 layers largely preserves performance, whereas further truncation causes substantial degradation, with accuracy drop-

Preprint. Work in progress.

ping from 84.3% to 72.8% and 61.5% at 16 and 12 layers, respectively. This highlights the limitation of fixed-depth inference, which imposes the same computational budget on all inputs. Yet, not every decision requires the same amount of computation. This motivates adaptive depth allocation (Figure 1, right), which allows computation to stop once sufficient depth is reached, reducing inference cost while preserving accuracy.

We find that decision computation is concentrated in intermediate layers, while independently trained smaller models largely replicate the full model rather than develop specialized decision capabilities (Section 3). First, decisions emerge within a narrow band of middle layers: integrated-gradient attribution places 50% of the decision by layer 20 and 80% by layer 23. Consistently, ablating layers 24–29 leaves accuracy unchanged, while ablating layers 15–23 eliminates the decision margin (Section 3.1). Second, independently trained smaller models behave more like truncated full models than independent specialists: 89–92% of their correct decisions are already made by the full model, and their distinct wins exceed full-model seed disagreement by only 2–3 points, mostly on low-confidence decisions (Section 3.2). These findings show that training smaller models or truncating the backbone alone is insufficient. Instead, decision models should learn to make reliable decisions at multiple depths, adapting computation to input difficulty.

Based on these findings, we propose NESTONE, a framework that transforms an LLM into a multi-depth System One decision model. NESTONE jointly trains the full model and nested prefix sub-models with shared final layers and decision head, combining decision supervision and self-distillation from the full model. This enables a single training run to produce multiple exits within one checkpoint. At each exit, a lightweight gate reads the hidden state to determine whether further computation is needed before producing the decision. Our contributions are threefold: (i) We show that decision information is concentrated in intermediate layers and that independently trained smaller models largely reproduce, rather than specialize in, the full model's decisions; (ii) We propose NESTONE, a nested multi-depth training framework that combines decision supervision, self-distillation, and a hidden-state stopping gate for transfering from LLMs to System One Models efficiency; (iii) We demonstrate that NESTONE matches or outperforms independently trained models at every depth while reducing inference cost through adaptive early exiting, using a single training run to obtain multiple models.

2 Background and Related Work

System One Decision Models. System One models take a state, a typed question, and a set of options, and return a distribution over the options in a single forward pass without generating text (TypeSafe AI, 2026). Following Jev, a growing number of such models are built by adapting a pretrained LLM. Open recipes read the option-symbol logits of an instruction-tuned LLM at the last position, with no training, or add LoRA adapters and a decision head initialized from the language-model head (Geuter et al., 2026). Intern-Decision (InternLM Team, 2026) fully fine-tunes Qwen3.5 (Qwen Team, 2026) at 0.8B, 2B, and 4B parameters, Microsoft-Decision-1 is post-trained from Qwen3.5-9B (Microsoft, 2026), CLM trains a decision model contrastively from Qwen3-8B (Kwok et al., 2026), and CS1 trains a 4B model on 12 million clinical decisions (Zhang et al., 2026). Other work changes how options are read out. Jeva encodes the request and each option separately so that the decision is permutation-equivariant, can abstain, and can reuse option states across requests (Li and Wang, 2026); Prism-1 scores each option in its own attention block so that neither option order nor added rivals change the result (Teng et al., 2026); and Consider-then-Choose removes order dependence at inference without training (Huang et al., 2026).

Evaluating System One Models. A second line of work asks where System One models can replace LLMs. System1-Arena compares ten System One models with open and closed LLMs through a shared decision interface and finds that the fast models suffice for many single-step decisions but fall behind on long-horizon tasks (Chen et al., 2026). Other studies evaluate System One models as decision points in agent harnesses (Li, 2026), and test their transfer to low-resource languages (Fadli et al., 2026). These studies treat each model as a black box and measure what it outputs.

Layer Pruning. The deeper layers of LLMs are known to be partly redundant. Removing a block of late layers costs little on question answering (Gromov et al., 2024; Men et al., 2024), and merg-

ing or pruning layers followed by distillation recovers most of the original accuracy at a smaller size (Yang et al., 2024; Muralidharan et al., 2024; Xia et al., 2024). These methods produce one smaller model with one fixed depth, and they select layers by similarity or importance scores computed for generation rather than by where a decision is computed.

Existing work on System One models focuses on how to train an LLM into a decision model, whether through readout design, training data, or architecture, and on how the resulting models behave. Each of these models keeps the full depth of its backbone, and cost is reduced only by choosing a smaller backbone. To our knowledge, no prior work asks where inside the LLM the decision is computed, or what a smaller decision model learns relative to the full one. We address both questions first (Section 3). The answers then guide an efficient way to turn one LLM into a System One model with several depths: nested prefix training with self-distillation, and a gate that reads the hidden state before the readout is computed (Section 4).

3 Understanding Depth in System One Models

We investigate what enables an LLM to become a System One decision model by comparing the model before and after decision fine-tuning, along with smaller variants obtained by progressively removing layers from the fine-tuned model. Specifically, we ask two questions: (i) Where in the network are decisions computed (Section 3.1)? (ii) What do smaller models learn compared to the full model (Section 3.2)?

Setting. We study Qwen3.5-4B (Qwen Team, 2026), whose 32 layers we index 0–31, and Intern-Decision-4B (InternLM Team, 2026), a released decision model built on the same backbone. To establish a controlled reference, we re-train Qwen3.5-4B using the same training recipe as Intern-Decision-4B. We refer to this re-trained model as **full32**, which serves as the starting point and reference for all our models. Specifically, full32 is fully fine-tuned with cross-entropy on the option symbols (AdamW, learning rate 2×10^{-6} with cosine decay, 2 epochs, about 131K tokens per step) on 44,941 decisions drawn from public training splits (Zhang et al., 2015; Liu et al., 2024; Lin et al., 2023; Ji et al., 2023; Casanueva et al., 2020; FitzGerald et al., 2023; Clark et al., 2019) and deduplicated against all test states. It reaches $84.3 \pm 0.2\%$ over three seeds, compared with 90.0% for Intern-Decision-4B, whose larger training set covers the J-Hard and Typed benchmarks better. We consider two layer-cutting strategies: a middle cut, $\mathrm{M}k$, which retains layers $0, \ldots, k-3$ and the final two layers (30–31), and a tail cut, $\mathrm{T}k$, which retains only the first k layers $(0, \ldots, k-1)$.

3.1 The Decision Is Written at Mid Depth

When Does the Decision Become Readable? We examine when the decision-related information emerges among layers and when it becomes accessible to the model's own readout. At each layer, we extract the residual state at the readout position and evaluate it using two complementary methods. The *logit lens* (Nostalgebraist, 2020) applies the model's final normalization and output head to the intermediate state, measuring whether the model can already produce the correct answer at that depth. In contrast, a *linear probe* (Alain and Bengio, 2016), trained to predict the gold option from the same state, measures how much decision-relevant information is linearly recoverable, regardless of whether the model's own readout can access it. Thus, the gap between the probe and logit-lens accuracies indicates information that has been encoded in the residual stream but is not yet usable by the model's output head.

Figure 2(a,b) shows a consistent pattern across the base model and Intern-Decision-4B. At layer 15, the probe substantially outperforms the logit lens (76.1% vs. 26.2% for the base model; 77.9% vs. 38.3% for Intern-Decision-4B), indicating that decision information is already encoded but not yet accessible to the model's readout. This gap closes over layers 19–23: for Intern-Decision-4B, the probe reaches 90.1% at layer 20, matching its final accuracy (90.0%), while the logit lens rises from 64.4% at layer 19 to 89.4% at layer 23. The two readouts then differ by only 0.1 points (vs. 2.8 points for the base model). Neither improves beyond layer 23, suggesting that layers 24–31 add no linearly recoverable decision information. Fine-tuning raises the probe plateau by about seven points (89.5% vs. 83.2% at layer 23) without shifting the depth at which decisions become readable.

Which Layers Shape the Decision? Readability shows where the answer is available, not which layers build it there. We therefore decompose the

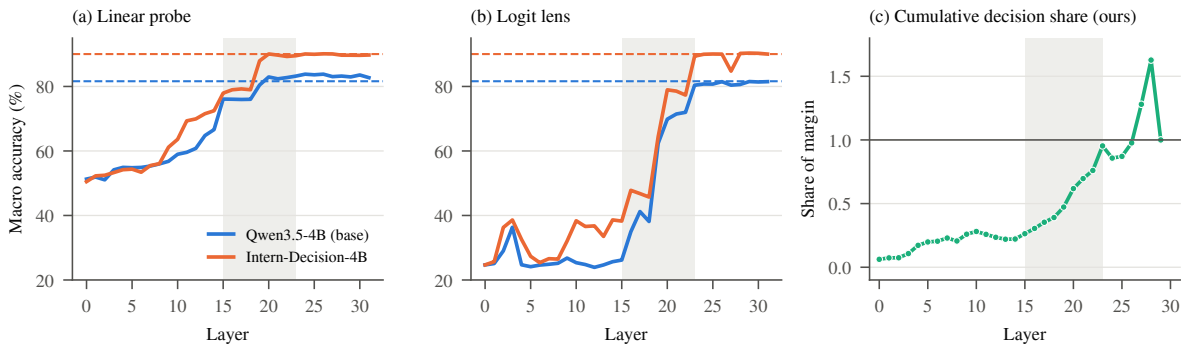

Figure 2: Where the decision becomes readable and where it is written. (a) Linear-probe accuracy for the gold option at the readout position of each layer, and (b) logit-lens accuracy using each model's own head, macro-averaged over the seven benchmarks; dashed lines mark each model's own accuracy. (c) Cumulative share of the decision margin contributed by layers up to ℓ in full32, measured with integrated gradients. The shaded band marks layers 15–23.

readout state, i.e. the residual stream at the readout position after the last body layer, as $\mathbf{h} = \mathbf{h}_0 + \sum_{\ell=0}^{29} \Delta_\ell$, where $\mathbf{h}_0$ is the token embedding and Δ_ℓ is the *contribution* of layer ℓ, i.e. the sum of the attention and MLP outputs that it adds to the residual stream. Let $s_o(\mathbf{h})$ be the logit of option o obtained by passing $\mathbf{h}$ through decision head. With gold option y among K options, the decision margin is

$$m(\mathbf{h}) = s_y(\mathbf{h}) - \frac{1}{K-1} \sum_{o \neq y} s_o(\mathbf{h}),$$

so a larger margin puts more probability on the gold option. Integrated gradients (IG) (Sundararajan et al., 2017) attribute the margin to these contributions. Along the straight path $\mathbf{h}(a) = \mathbf{h}_0 + a(\mathbf{h} - \mathbf{h}_0)$ from the embedding to the readout state, layer ℓ receives

$$\mathrm{IG}_\ell = \Delta_\ell^\top \int_0^1 \nabla_{\mathbf{h}} m\big(\mathbf{h}(a)\big)\, da,$$

its contribution multiplied by the average sensitivity of the margin along the path. A positive IG_ℓ means that layer ℓ moves the decision distribution toward the gold option, and a negative one moves it away. The attributions add up to the total margin change, $\sum_\ell \mathrm{IG}_\ell = m(\mathbf{h}) - m(\mathbf{h}_0)$.

Figure 2(c) reveals three distinct regimes in full32. Early layers (0–14) account for only 22% of the final margin, with no individual layer contributing more than 7%. Middle layers (15–23) contribute another 73%, raising the cumulative margin from 22% to 95%. In contrast, late layers exhibit substantial but largely canceling updates: layers 27 and 28 contribute 30% and 35%, respectively, raising the running margin to 1.6× its final value, before layer 29 reverses this increase (−63%). Consequently, layers 24–29 contribute only 5% net, primarily rescaling the margin without changing the preferred option. Overall, the middle layers play the dominant role in shaping the final decision distribution of a fine-tuned System One model.

(a) Removing layers (Intern-Decision-4B, 90.05)

Cut	Removed	Acc	Δ [95% CI]
M26	24–29	90.1	+0.0 [−0.9, +0.9]
T24	24–31	89.4	−0.6 [−1.2, −0.1]
M24	22–29	88.8	−1.2 [−2.1, −0.4]
T22	22–31	78.5	−11.6 [−14.1, −9.3]

(b) Mean ablation (full32)

Ablated	Margin	ΔAcc
0	95%	−46.6
6	31%	−7.8
24–29	−6%	+1.7
20–23	34%	−2.3
19 + 23	53%	−5.5
15–23	102%	−57.8

Table 1: (a) Accuracy after removing layers without retraining. (b) Share of the mean decision margin removed, and accuracy change, when the contribution of a single early layer (0, 6) or jointly of a group of layers is replaced by its mean (348 decisions, 50 per benchmark).

Which Layers Can Be Removed? Table 1 evaluates layer importance through two complementary interventions. First, we remove layers without retraining (Table 1a), using a middle cut Mk or a tail cut Tk. Second, we perform *joint mean ablation* (Table 1b): for each layer ℓ, we estimate its mean contribution $\bar{\Delta}_\ell$ across token positions and approximately 1,700 decisions (up to 400 per benchmark), then replace each layer's contribution with this

mean at every position. This preserves the residual stream's typical scale while removing decision-specific information. Joint ablation measures the group's collective contribution to decision-making, accounting for interactions among its layers while preventing them from compensating for one another. We report the resulting change in mean margin and accuracy over 348 decisions.

The two interventions reveal distinct layer roles. In Intern-Decision-4B, removing layers 24–29 without retraining leaves accuracy unchanged, while removing layers 24–31 costs only 0.6 points. Similarly, full32 retains 84.4% accuracy under T24, compared with 84.2% uncut; cutting into the middle band is substantially more damaging, with T22 losing 11.6 points. Mean ablation confirms this pattern: jointly ablating layers 24–29 slightly improves margin and accuracy, whereas ablating layers 15–23 eliminates the entire margin and reduces accuracy by 57.8 points. Early layers, however, remain essential despite their small direct margin contributions: ablating layer 0 costs 46.6 points and layer 6 costs 7.8, reflecting their role in constructing features used by later layers. Consistent with this, a 16-layer model trained only on layers 8–21 and 30–31, the layers most affected by fine-tuning, achieves just 35.7%, compared with 73.8% for the M16 prefix model.

Takeaway. A decision-tuned LLM writes its decision in a band of middle layers (15–23) on top of feature-building early layers, and its last body layers carry no decision.

3.2 Diagnosing Decision Behavior

We next examine what independently trained smaller models learn relative to full32. We fine-tune M24, M16, and M12 using the same data and recipe as full32, obtaining 83.5%, 73.8%, and 62.9% accuracy, respectively. We compare their predictions with full32 decision by decision, categorizing each as both correct, only full32 correct, only the smaller model correct, or both incorrect. To establish a noise floor, we also compare full32 against two independently trained seeds (43 and 44), which uniquely solve 1.4–2.0% of decisions missed by the reference seed (Table 2, top).

Smaller Models Mostly Reproduce Full32's Decisions. Table 2 shows that M24 rarely makes a correct prediction that full32 misses: 1.9% of decisions, within the noise floor of 1.4–2.0%, and its subset score (.967) matches that of another full32 seed (.968–.977). M16 and M12 remain largely subsets of full32: 91% and 89% of their correct predictions, respectively, are also correct under full32. Moreover, they lose substantially more decisions than they uniquely solve, by roughly a factor of three to four (1,012 vs. 352 for M16; 1,513 vs. 425 for M12).

		Only correct				Acc | p_{full}	
Model	Acc	full	small	Sub.	Rec.	$\geq.8$	$<.2$
Noise floor: other seeds of full32							
seed 43	84.1	1.5	1.4	.977	.977	100.0	0.7
seed 44	84.5	1.8	2.0	.968	.972	100.0	0.9
T24 cut	84.5	1.5	1.7	.975	.975	99.9	1.7
M24	83.5	2.7	1.9	.967	.959	99.7	2.0
M16	73.8	14.4	3.8	.914	.795	84.1	16.7
M12	62.9	25.6	4.3	.891	.666	70.4	19.5

Table 2: Decision-level comparison with full32 (seed 42). **Noise floor**: full32 trained with seeds 43 and 44, compared in the same way. **Only correct**: share of decisions answered correctly by only full32 or the smaller model. **Sub.** and **Rec.**: $P(\text{full correct} \mid \text{small correct})$ and $P(\text{small correct} \mid \text{full correct})$, respectively. **Acc** $\mid p_{\text{full}}$: smaller-model accuracy when full32 assigns the gold option probability ≥ 0.8 or < 0.2. M24, M16, and M12 are independently trained; T24 is truncated from full32 without retraining. All rates are macro-averaged across benchmarks.

Unique Correct Predictions Are Rare and Uncertain. M16 and M12 uniquely solve only 3.8–4.3% of decisions, just 2–3 points above the noise floor. These wins are low-confidence (median confidence 0.59–0.61; full32 gold probability 0.23–0.33) and concentrated in Typed and AGNews (78%), with none in J-Easy or J-Orig. Noise-floor disagreements show the same pattern: median confidence of 0.52–0.55, with 71–73% from Typed and AGNews.

Smaller Models Also Lose Reliable Decisions. When full32 assigns the gold option probability ≥ 0.8, M24, M16, and M12 achieve 99.7%, 84.1%, and 70.4% accuracy, respectively, while other full32 seeds reach 100%. When full32 is confidently wrong ($p < 0.2$), smaller models recover only 17–20% of these decisions, compared with 0.7–0.9% for other seeds. Their apparent gains therefore reflect greater disagreement with full32, rather than systematic correction of its errors. Overall, smaller models reproduce many of full32's correct decisions but lose substantial reliable capability without consistently correcting its mistakes.

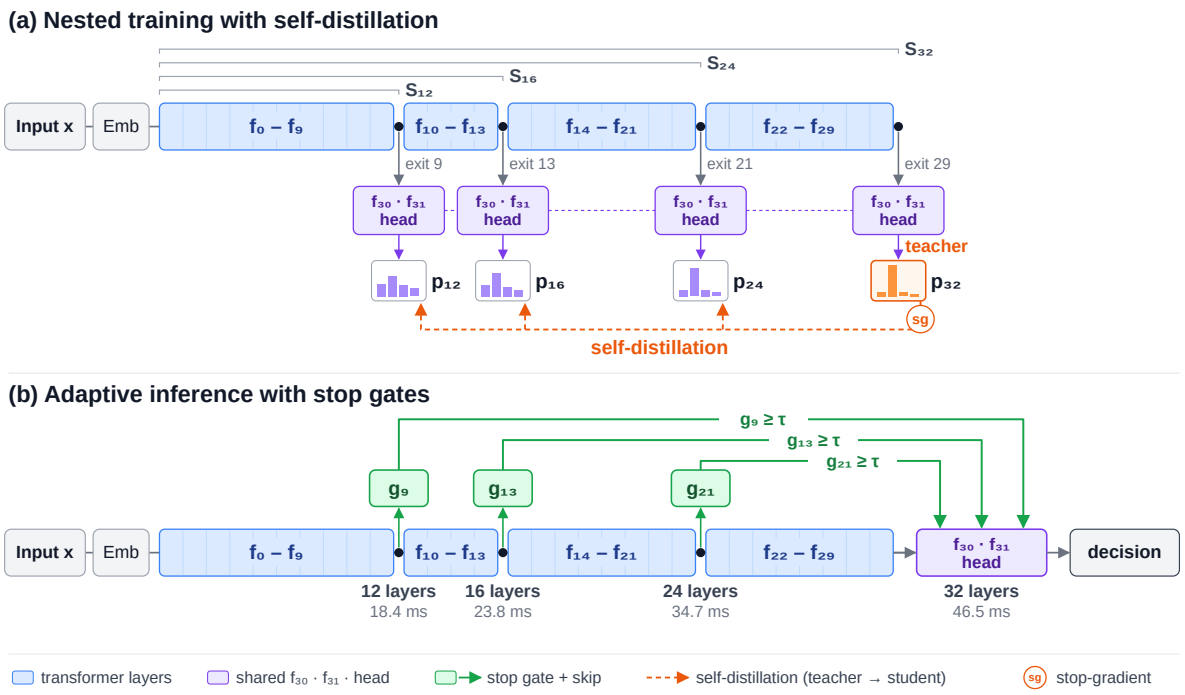

Figure 3: **Overview of NESTONE. (a) Nested training with self-distillation.** Prefix exits share the full model's weights and final block. The full model learns from the gold option y, while each sub-model learns from both y and the full model's distribution. **(b) Adaptive inference with stop gates.** The model runs layer by layer, with a lightweight gate evaluating the hidden state at each intermediate exit. If $g_e \geq \tau$, it runs the final block and returns a decision; otherwise, it continues from the saved states without recomputation.

Takeaway. Smaller decision models trained on their own do not learn a different solution; they learn a degraded version of the full model's. This suggests training them as *parts* of the full model, sharing its weights and using its own output distribution as a target, so that each smaller model inherits as much of the full model's skill as its depth allows.

4 Methodology

Section 3 shows that intermediate layers already encode sufficient information to identify the correct option, while smaller decision models are best viewed as prefixes of the full model that should learn to imitate it. Building on these findings, NESTONE trains nested prefix sub-models within the full model using self-distillation to facilitate information sharing across depths (Sections 4.1 and 4.2). A lightweight gate then selects the exit depth for each decision based on question difficulty (Section 4.3).

4.1 Prefix Sub-Models with a Shared Coda

Let $f_0, \ldots, f_{31}$ be the layers of the decision model, and let $\mathbf{h}^{(e)} = f_e \circ \cdots \circ f_0(x)$ be the residual stream after layer e. We call the last two layers together with the decision head the *coda* $C = \text{head} \circ f_{31} \circ f_{30}$. An *exit* after layer e reads out the decision distribution

$$p_e(\cdot \mid x) = \text{softmax}\big(C(\mathbf{h}^{(e)})_{\mathcal{O}}/T_e\big),$$

where $\mathcal{O}$ is the set of option symbols and T_e is a temperature fitted per exit. The sub-model S_k with k layers uses the exit after layer $e_k = k-3$, eg. S_{12}, S_{16}, and S_{24} exit after layers 9, 13, and 21, and S_{32} is the full model. This design follows Section 3.1. Prefixes keep the early feature-building layers that no smaller model can do without, and S_{24} reaches into the decision band while dropping the late layers that carry no decision. The coda is shared so that every exit reads out through the same final layers and head. Because each sub-model is a prefix of the next, the states computed for a shallow exit are reused when a decision continues to a deeper exit, so stopping early never requires recomputation.

4.2 Nested Training Objective

Following Section 3.2, we train all sub-models jointly with the full model, so that they share its weights and learn from its output. Let y be the gold option and p_k the distribution of S_k. At each step

we compute

$$\mathcal{L} = \mathrm{CE}(p_{32}, y) + \sum_{k \in \mathcal{K}_t} \Big[(1 - \alpha_t)\, \mathrm{CE}(p_k, y) + \alpha_t\, \mathrm{KL}\big(\mathrm{sg}(p_{32}) \,\|\, p_k\big) \Big],$$

where sg stops the gradient and the KL term is taken over the options of the decision. The full model is trained only on the gold option, and each sub-model learns both from the gold option and from the full model's distribution (Hinton et al., 2015). This self-distillation term makes each sub-model imitate the full model, as Section 3.2 suggests. We linearly increase α_t from 0 to 0.5 over the first half of training, so that the full model's distribution becomes a target only once it is informative. The full model serves as its own teacher, so NESTONE needs no external decision signal. All parameters are fine-tuned with the recipe of full32.

4.3 Choosing Depth per Decision

A single checkpoint now offers exits at intermediate depths, as well as the full model. To choose an exit for each decision, we attach a lightweight gate to each intermediate exit. The gate reads the hidden state $\mathbf{z}_e$ at the readout position immediately after the corresponding middle layer:

$$g_e(x) = \sigma\big(\mathbf{w}_e^\top \mathbf{z}_e + b_e\big),$$

where σ is the sigmoid function. Each gate is a logistic regression model trained to predict whether its corresponding exit answers correctly, using labels $c_e = \mathbf{1}[\arg\max p_e = y]$ and class-balanced weights. At inference, the model runs layer by layer. At each intermediate exit, it stops if $g_e(x) \geq \tau$, runs the coda, and returns $\arg\max p_e$; otherwise, it continues from the saved states. The final exit always answers.

Answering at exit e costs $e + 3$ layers: the prefix plus the coda. Because the gate reads the hidden state before the coda, evaluating it requires only a dot product and a sigmoid operation. A single threshold τ is shared across exits and selected on held-out data as the most aggressive value for which macro accuracy remains within δ points of always exiting at the final intermediate depth.

5 Experimental Results

We evaluate NESTONE on three questions: (i) Does one nested checkpoint match models trained separately at each depth (Section 5.1)? (ii) Can the gate choose depth per decision without losing accuracy (Section 5.2)? (iii) Why does nested training help (Section 5.3)?

Layers	Separate	NESTONE	Δ [95% CI]
32	$\mathbf{84.3}_{\pm 0.2}$	$84.0_{\pm 0.6}$	-0.3 [-1.3, $+0.8$]
24	$83.5_{\pm 0.8}$	$\mathbf{83.7}_{\pm 0.3}$	$+0.2$ [-1.1, $+1.5$]
16	$72.8_{\pm 0.9}$	$\mathbf{74.7}_{\pm 0.9}$	$+1.8$ [-0.1, $+3.8$]
12	$61.5_{\pm 1.5}$	$\mathbf{64.4}_{\pm 0.4}$	$\mathbf{+2.9}$ [$+0.7$, $+5.3$]

Table 3: Benchmark average (%) of separately trained models and of the exits of one NESTONE checkpoint (mean$_{\pm \mathrm{sd}}$ over 3 seeds). Δ: NESTONE minus separate, with a CI that resamples decisions and seeds.

Setup. All models start from Qwen3.5-4B and use the data, recipe, and benchmark of Section 3. We compare NESTONE with full32, M24, M16, and M12, each trained separately on its own. Every model, nested or separate, is trained with three seeds, which change the data order and, for NESTONE, which sub-models are sampled. One NESTONE run takes 80 minutes on a single H200, against 94 minutes for training the four separate models. At batch size one, the 12-, 16-, 24-, and 32-layer exits take 18.4, 23.8, 34.7, and 46.5 ms per decision. For early exit, we count cost as the mean number of layers run per decision, including the coda. We fit each gate and its threshold on one benchmark-stratified half of the benchmark together with the calibration split, and report results on the other half. Each half is used once for testing.

5.1 One Checkpoint versus Separate Models

One Checkpoint Matches or Outperforms Separate Models at Every Depth. Table 3 compares the four exits of NESTONE with independently trained models at matching depths. No exit performs significantly worse, while the shallower exits achieve higher accuracy. The largest gain occurs at 12 layers (64.4% vs. 61.5%, +2.9 points), with a confidence interval excluding zero even when resampling seeds. At 16 layers, the gain is 1.8 points but does not reach significance. The 24-layer exit achieves 83.7%, comparable to M24 (+0.2 points) and full32 (-0.6 points), while using 25% fewer layers and reducing latency from 46.5 to 34.7 ms. At 32 layers, accuracy is only 0.3 points below full32, indicating that joint training imposes little cost on the full model. Thus, a single 80-minute training run replaces four separate runs while yielding the best accuracy at 12, 16, and 24 layers.

Rule	δ	ΔAvg	ΔL4	Mean layers All	Easy	Hard
Fixed exit 13	–	-9.89	-4.49	16.0	16.0	16.0
Fixed exit 21	–	0.00	0.00	24.0	24.0	24.0
Max-prob	0.25	-0.30	-0.04	23.8	23.4	27.8
Confidence gate	0.25	-0.39	-0.06	23.2	22.3	27.5
State + confidence	0.25	-0.26	$+0.11$	23.4	19.7	27.9
State gate (ours)	0.25	-1.56	$+0.08$	**21.6**	20.2	**24.0**
Max-prob	2	-1.35	-0.12	21.3	19.4	26.9
Confidence gate	2	-1.36	-0.13	21.1	19.0	26.6
State + confidence	2	-1.37	-0.09	20.2	**15.4**	27.3
State gate (ours)	2	-1.68	$+0.03$	**20.2**	17.5	**23.9**

Table 4: Early exit over exits 9, 13, and 21 of NESTONE (seed 42). ΔAvg: change in benchmark average relative to always exiting after layer 21 (83.8%). ΔL4: accuracy change on the four largest benchmarks (AGNews, WildJailbreak, Typed, J-Hard; 96% of decisions), weighted by decisions. Mean layers include the coda and every readout computed. Easy: AGNews and WildJailbreak; hard: Typed and J-Hard. Max-prob stops when the calibrated top probability clears the threshold; the confidence gate is a logistic regression on top probability, margin, and entropy.

5.2 Choosing Depth per Decision

The Hidden State Predicts Whether an Exit Is Correct. We first measure how well each signal separates correct from wrong answers at an exit, using AUROC on the test halves. The state gate reaches 0.86, 0.89, and 0.88 at exits 9, 13, and 21, against 0.85, 0.86, and 0.84 for the calibrated top probability, and 0.86, 0.86, and 0.84 for the confidence gate that combines top probability, margin, and entropy. The hidden state thus carries more information about correctness than the exit's own confidence, especially at exit 13 and 21, and it is available before the coda is computed.

Stopping Rules. We compare the state gate with three rules based on the exit distribution p_e: *Max-prob*, which uses the calibrated top probability; *confidence gate*, a logistic regression on the top probability, top-two margin, and entropy; and *State + confidence*, which combines these features with the hidden state $\mathbf{z}_e$. All rules use the same training labels, class-balanced weights, and fitting data. They evaluate exits 9, 13, and 21 in order, stopping at the first score above τ or answering at exit 21 otherwise; τ is selected as in Section 4.3. The key difference is inference cost: distribution-based rules must run the coda to obtain p_e, adding two layers at each checked exit, whereas the state gate reads $\mathbf{z}_e$ before the coda and runs it only once, at the selected exit. Fixed exits 13 and 21 serve as references.

The State Gate Gives the Best Trade-off. Table 4 compares the gating rules at two accuracy thresholds. The state gate achieves the lowest average depth at both $\delta = 0.25$ and $\delta = 2$, using 21.6 and 20.2 layers, respectively, compared with 24 layers for the fixed exit. On hard benchmarks, it uses 23.9–24.0 layers, whereas confidence-based rules incur two additional layers per readout, resulting in an average depth of 26.6–27.9 layers. On the four largest benchmarks, which account for 96% of decisions, the state gate preserves the fixed-exit accuracy, with changes of only $+0.08$ and $+0.03$ points at the two thresholds. The other rules also remain within 0.13 points of the fixed exit, but achieve savings differently: confidence-based rules save computation on easy decisions while spending more on hard ones, whereas the state gate reduces depth on easy decisions (to 20.2 and 17.5 layers) without incurring additional computation on hard decisions. Randomly mixing exits 13 and 21 to match the same average depth instead reduces accuracy by 3.0 and 4.7 points.

The Accuracy Drop Is Concentrated on Small Benchmarks. The state gate's macro-accuracy drop (-1.56 and -1.68 points) comes almost entirely from the three smallest benchmarks, which contain only 430 of 12,351 decisions but account for three of seven macro-averaged benchmarks. At $\delta = 0.25$, it misses seven additional decisions on these benchmarks but loses none on larger ones. Compute savings depend on traffic mix: average depth ranges from 24 layers on hard-only traffic to 20.2 on easy-only traffic (22.1 for an even mix), and to 17.5 layers at $\delta = 2$ (20.7 for an even mix; Figure 1, right). Since the selected threshold varies across training splits (0.80 vs. 0.92), it should be calibrated on held-out deployment-like traffic. Adding the full model as a final exit further improves the trade-off: at $\delta = 2$, the gate matches its accuracy on the four largest benchmarks (-0.01 points) while using 23.9 layers on average, 19.8 on easy decisions, and 31.3 on hard ones.

5.3 Why Nested Training Helps

Shallow Exits Keep More of the Full Model's Skill. Section 3.2 found that separately trained small models are degraded copies of the full model. Table 5 shows that nesting reduces this degradation rather than adding new skills. At 12 layers, the

		Only correct			Acc | p_{full}	
Model	Acc	full	small	Rec.	≥.8	<.2
full32, seeds 43/44	84.3	1.6	1.7	.975	100.0	0.8
M12	61.5	26.5	3.8	.657	69.5	15.8
NESTONE 12	64.4	**23.4**	3.6	**.688**	**72.8**	14.8
M16	72.8	15.1	3.6	.785	83.3	17.3
NESTONE 16	74.7	**13.2**	3.5	**.810**	**86.0**	17.9
M24	83.5	2.9	2.2	.956	99.4	3.8
NESTONE 24	83.7	3.4	2.8	.950	98.8	3.4
NESTONE 32	84.0	2.7	2.4	.961	99.3	2.8

Table 5: Decision-by-decision comparison with full32 (seed 42), as in Table 2, averaged over 3 seeds of each model. NESTONE k is the k-layer exit.

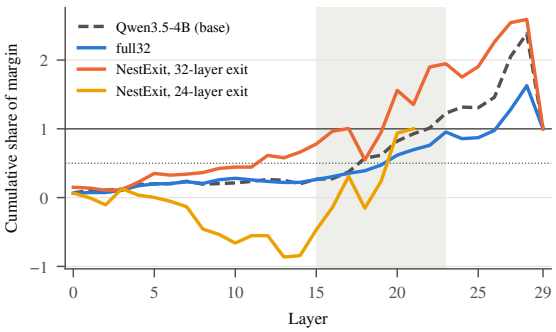

Figure 4: Cumulative share of the decision margin contributed by layers up to ℓ, measured with integrated gradients on 348 decisions. The 24-layer exit of NESTONE reads out after layer 21. The shaded band marks layers 15–23.

exit wins no more decisions of its own than M12 does (3.6% vs. 3.8%). Instead, it loses fewer of full32's correct decisions (23.4% vs. 26.5%), so its recall of full32's skill rises from 0.66 to 0.69. On decisions full32 is confident about, it is right 72.8% of the time, against 69.5% for M12. The 16-layer exit shows the same pattern: recall rises from 0.79 to 0.81, and accuracy on confident decisions from 83.3% to 86.0%, with no more wins of its own (3.5% vs. 3.6%). This is the effect the self-distillation term targets: the shallow exit imitates the full model and keeps more of what the full model knows.

Deep Exits Stay Close to the Full Model. At 24 and 32 layers, the nested exits keep as much of full32's skill as M24 (recall 0.95–0.96 vs. 0.96), and their own wins (2.4–2.8%) are close to those of M24 (2.2%). Nesting neither adds nor removes skill at these depths. This matches Table 3, where the deep exits tie their separate counterparts. The benefit of nesting at depth is therefore cost, not accuracy: one checkpoint provides the 24-layer exit and the full model together, and the gate can choose between depths for each decision.

Where Does NESTONE Write the Decision? Figure 4 shows that nested training shifts decision formation toward earlier layers, reversing the delay caused by plain fine-tuning. In the 32-layer exit, layers 0–16 account for half of the final margin (bootstrap range 15–16), and layers 0–20 account for 80% (19–20), compared with layers 20 and 23 in full32. The first 20 layers contribute 77% of the margin, versus 47% in full32, so the full model largely commits to its decision within the prefix shared by the 24-layer exit. Later layers mainly rescale this decision: their cumulative contribution peaks at 2.5× the final margin before layer 29 removes the excess. Jointly ablating layers 24–29 increases the margin by 4% and reduces accuracy by only 0.6 points. The 24-layer exit follows a similar pattern: layers 15–21 form its decision, with layer 20 alone contributing 48%. Thus, nested training concentrates decision formation in layers shared across deeper exits.

6 Conclusion

We studied adaptive computation for System One decision-making in LLMs. Our analysis reveals that decisions primarily emerge in middle layers, with later layers contributing little additional information, and that smaller independently trained models largely replicate the full model's decisions. Motivated by these findings, NESTONE jointly trains nested prefix sub-models via self-distillation and employs a hidden-state gate for adaptive early exiting. Across three seeds, a single checkpoint matches or outperforms independently trained models at all exit depths, improving 12-layer accuracy by 2.9 points and approaching the 32-layer model within 0.6 points at 24 layers. Adaptive exiting further reduces computation by 16–27% on easy decisions without sacrificing accuracy, while retaining fixed-depth exits for harder cases. These results highlight the benefits of nested training for aligning shallow exits with full-model decisions. Future work will investigate end-to-end latency, gate calibration under real-world traffic, and scaling to larger models and finer-grained exits.

References

Guillaume Alain and Yoshua Bengio. 2016. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*.

Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. 2020. Efficient intent detection with dual sentence encoders. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*.

Tianyu Chen, Xinyang Liu, Ruyi An, Zhendong Wang, and Mingyuan Zhou. 2026. System1-Arena: Which decisions can be left to system 1? *arXiv preprint arXiv:2610.21244*.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*.

Romi Fadli, Rusdah, Wahyu Ramadani Putri Utami, and Zulfa Malika Amana. 2026. Does system one transfer? calibrated typed-decision models, label budgets, and translation for Indonesian regional languages. *arXiv preprint arXiv:2610.14112*.

Jack FitzGerald et al. 2023. MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.

Jonathan Geuter, Andrew Cheng, and David Alvarez-Melis. 2026. Releasing the inner Jev: A simple recipe for open decision models. *arXiv preprint arXiv:2610.17205*.

Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. 2024. The unreasonable ineffectiveness of the deeper layers. *arXiv preprint arXiv:2403.17887*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Jianguo Huang, Yao Long Teng, Yanchen Deng, and Bo An. 2026. Consider, then choose: Order-invariant decisions in system one models. *arXiv preprint arXiv:2610.18233*.

InternLM Team. 2026. Intern-Decision: Multimodal structured decision models. https://github.com/internlm/Intern-Decision. Model weights: https://huggingface.co/collections/internlm/intern-decision.

Jiaming Ji et al. 2023. BeaverTails: Towards improved safety alignment of LLM via a human-preference dataset. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

Daniel Kahneman. 2011. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York.

Jacky Kwok, Hangoo Kang, Tarun Suresh, Jon Saad-Falcon, Marco Pavone, Christopher Ré, and Azalia Mirhoseini. 2026. Contrastive language models: A system one model for fast and generalizable decision-making. Blog post, https://contrastive-lm.notion.site.

Jiawei Li. 2026. Fast models, slow evidence: A paired and self-audited evaluation of system-1 decision models for LLM agent harnesses. *arXiv preprint arXiv:2610.02267*.

Qi Li and Xinchao Wang. 2026. Jeva: System one models from first principles. *arXiv preprint arXiv:2610.18773*.

Zi Lin et al. 2023. ToxicChat: Unveiling hidden challenges of toxicity detection in real-world user-AI conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.

Weiwen Liu et al. 2024. ToolACE: Winning the points of LLM function calling. *arXiv preprint arXiv:2409.00920*.

Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. 2024. ShortGPT: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*.

Microsoft. 2026. Introducing Microsoft-Decision-1, our model for fast decision-making. https://commandline.microsoft.com/microsoft-decision-1-model-foundry/.

Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. 2024. Compact language models via pruning and knowledge distillation. In *Advances in Neural Information Processing Systems*.

Nostalgebraist. 2020. Interpreting GPT: The logit lens. LessWrong blog post.

Qwen Team. 2026. Qwen3.5: Towards native multimodal agents.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning*.

Yao Long Teng, Jianguo Huang, Yanchen Deng, Jiayi Cai, and Bo An. 2026. Prism-1: Position- and rival-independent scoring for system one models. *arXiv preprint arXiv:2610.22066*.

TypeSafe AI. 2026. Introducing system one models and Jev. https://typesafe.ai/blog/introducing-system-one-models-and-jev.

Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. 2024. Sheared LLaMA: Accelerating language model pre-training via structured pruning. In *International Conference on Learning Representations*.

Yifei Yang, Zouying Cao, and Hai Zhao. 2024. LaCo: Large language model pruning via layer collapse. In *Findings of the Association for Computational Linguistics: EMNLP 2024*.

Andrew Zhang, Trinh T. L. Vuong, Sophia J. Wagner, Dongmin Bang, Tong Ding, Yuchang Su, Mary Cipperman, Christina S. Chen, Judith Lefkes, Ramin Kahidi, Ming Y. Lu, Caiwei Tian, Tianhao Luo, Jingnan Ma, Rowland Pettit, Long Phi Le, and Faisal Mahmood. 2026. A general-purpose system one decision model for clinical records. *arXiv preprint arXiv:2610.27099*.

Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*.